2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133357

Late Breaking Results:

On-the-Fly Hadamard Hypervector Processing for Efficient Hyperdimensional Computing

Abu Kaisar Mohammad Masum*, Mehran Shoushtari Moghadam§, Sabrina Hassan Moon†, Ahmed Mamdouh Mohamed Ahmed†, M. Hassan Najafi§, Dayane Reis†, and Sercan Aygun*
*School of Computing and Informatics, University of Louisiana at Lafayette, Lafayette, LA, USA
§Electrical, Computer, and Systems Engineering Department, Case Western Reserve University, Cleveland, OH, USA
†Bellini College of AI, Cybersecurity and Computing, University of South Florida, Tampa, FL, USA
{c00591145, sercan.aygun}@louisiana.edu, {moghadam, najafi}@case.edu, {ms38, ahmed749, dayane3}@usf.edu

***Abstract*—Inspired by the human brain, Hyperdimensional Computing (HDC) processes information efficiently by operating in high-dimensional space using hypervectors. While previous works focus on optimizing pre-generated hypervectors in software, this study introduces a novel on-the-fly vector generation method in hardware with $O(1)$ complexity, compared to the $O(N)$ iterative search used in conventional approaches to find the best orthogonal hypervectors. Our approach leverages Hadamard binary coefficients and unary computing to simplify encoding into *addition-only* operations after the generation stage in ASIC, implemented using in-memory computing. The proposed design significantly improves accuracy and computational efficiency across multiple benchmark datasets.**

I. Introduction

Hyperdimensional Computing (HDC) is an emerging paradigm that draws inspiration from how the human brain works [1]. HDC operates in a hyperdimensional space using hypervectors ($\mathcal{HV}$). $\mathcal{HV}$s can typically have a high dimensionality of thousands of random bits (e.g., D=10,000) for better *orthogonality*. Information is encoded in either a *binary* format ('0' and '1') in physical hardware, or a *bipolar* format ('-1' and '$+1$'). To ensure *accuracy* and *efficiency* in HDC models, $\mathcal{HV}$s are generated through multiple iterations of randomization to maintain orthogonality. After the optimized *vector generation* (GEN) with N-iteration for improved orthogonality, HDC encoding involves two key operations: ❶ *binding* (BIN) and ❷ *bundling* (BUN) [2]. ❶ *Binding* combines $\mathcal{HV}$s using XOR (binary) or multiplication (bipolar) to create unique representations. ❷ *Bundling* merges $\mathcal{HV}$s by popcount (binary) or addition (bipolar) to capture shared features.

Traditional pseudo-random mechanisms, often producing low-quality results [3], have been used to generate orthogonal $\mathcal{HV}$s through N iterations, relying heavily on software platforms. Adopting quasi-random sources (e.g., Sobol) has enhanced accuracy and broadened HDC's applicability across various systems [4]. Fig. 1(a) depicts the baseline HDC architecture. $\mathcal{HV}$s are generated using pseudo-random sources (e.g., LFSR: linear-feedback shift register). In the generation phase (GEN), each scalar (e.g., pixel intensity in step ①) or symbol (e.g., pixel position with an assigned threshold in step ②) is compared with random numbers to obtain $\mathcal{HV}$s. Step ③ shows the primary encoding step, where two feature $\mathcal{HV}$s are combined using XOR (BIN). In step ④, the XORed $\mathcal{HV}$s are accumulated (BUN) to produce a single $\mathcal{HV}$ for holistic representation. Finally, in step ⑤, the summed $\mathcal{HV}$ is binarized, incorporating contributions from all samples of the same class in the training data. This yields the final binary class $\mathcal{HV}$ (C).

II. Proposed AddOn-IHD

Utilizing LFSR in the baseline HDC has two major drawbacks: (i) It relies on pseudo-randomness, where a single run may not guarantee optimal accuracy, and (ii) it requires two separate encodings with multiple LFSRs and XOR operations, leading to high implementation costs. This work introduces **AddOn-IHD** (Addition-Only In-Memory HyperDimensional), a framework that simplifies vector generation and enables on-the-fly processing for HDC. Processing $\mathcal{HV}$s on-edge without pre-processing on software offers: i.) End-to-end HDC design, and ii.) adaptability to varying count vectors (e.g., dynamic training image sizes), making on-the-fly vector processing more efficient and adaptable for real-world HDC systems. The proposed vector generation leverages a quasi-orthogonal Hadamard matrix, replacing multiplications with simple wiring (or inversion). To the best of our knowledge, this is the first time in the literature that Hadamard matrix [5] is used to transform and manipulate $\mathcal{HV}$s in HDC. Hadamard coefficient processing eliminates $O(N)$ time-complex pseudo-randomness, providing the needed orthogonality for HDC with $O(1)$ complexity. Fig. 1(b) illustrates the overall proposal and computing architecture. Compared to the baseline design in Fig. 1(a), **AddOn-IHD** simplifies the GEN process by replacing LFSRs with an up-counter for unary bit-stream generation [6] (step ①) and removing two-way data encoding (step ②). **AddOn-IHD** eliminates XOR operations by *wiring-negating* $\mathcal{HV}$ via Hadamard (BIN, step ③). The proposed framework retains these three steps in ASIC design, where the Hadamard-based $\mathcal{HV}$s are stored in resistive RAM (ReRAM) for in-memory computing (IMC). **AddOn-IHD** simplifies the overall encoding process by reducing it to *addition only*, enabling efficient accumulation within IMC. In contrast, the baseline design requires handling XOR operations within memory, increasing complexity. **AddOn-IHD** eliminates this overhead, streamlining IMC integration. Additionally, it removes the reliance on multiple LFSR runs for performance optimization, introducing a single deterministic process. This eliminates the need for precomputing vectors in software, allowing the entire process to be executed independently in hardware.

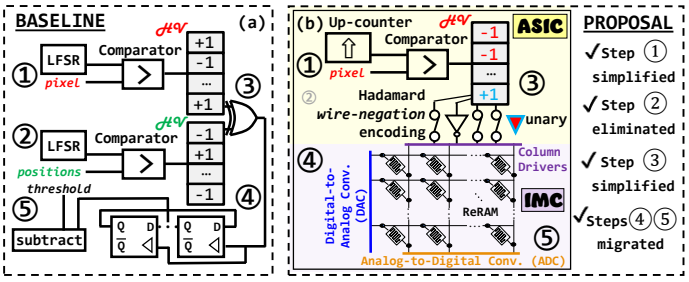

Fig. 1. (a) Baseline HDC vs. (b) **AddOn-IHD**.

Fig. 2 details the workflow of the proposed framework with unary computing basics. Unary bit-streams represent scalar values (such as *pixel* intensities) using uniform D-sized binary vectors, eliminating significant bit positions [6]. This representation groups '1's together to reduce toggling and improve energy efficiency [3]. Fig. 2(a) shows an example with D=8-bit unary bit-streams for scalar values in the $[-1, +1]$ interval; e.g., *0.75* is represented by 11111110 or *0* is encoded as 11110000 (half the bits are logic 1s, and the other half are 0s). The *0* scalar value is important for the *sign threshold*; A majority of 1s indicates the $+$ sign, while a majority of 0s indicates the $-$ sign. In HDC, binarization after additions can be obtained by simply checking the *majority* of binary values. In unary computing, negation is a simple inverter, and the sign can easily be altered, as shown in Fig. 2(a) [6].

The **AddOn-IHD** framework operates solely on raw input data (e.g., *pixel* intensities) without requiring additional feature extraction, as shown in Fig. 2(b). To preserve the positional information of the data (such as *pixel positions* in *image data* or *cell positions* in *tabular data*), we leverage the Hadamard matrix. A Hadamard is a square matrix whose entries are either -1 or $+1$, and the rows and columns are orthogonal to each other. In **AddOn-IHD**, the Hadamard matrix is used for processing the pixel intensity unary vectors. Each unique Hadamard entry (a cell from the matrix size of $m \times n$) positionally

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.

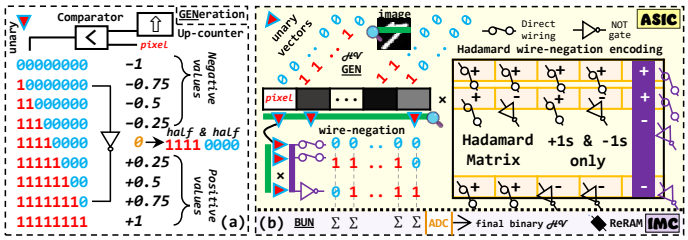

Fig. 2. (a) Unary computing basics, (b) **AddOn-IHD** framework workflow.

matches with the single pixel of the processing image (size of $m \times n$). When the Hadamard enrty includes -1, a *sign change* is implemented by inverting unary pixel vector using a NOT gate. Direct wiring is applied to a unary vector for *no change*, when the Hadamard value is $+1$. We call this approach *Hadamard wire-negation encoding*. This deterministic process is hardcoded in ASIC via wires & NOT gates, as shown in Fig. 2(b).

The wired or negated vectors are accumulated similarly to the baseline HDC, where each bit of a unary pixel vector is summed column-wise with other vectors using a pop-count ($\sum$), as illustrated in Fig. 2(b). A ReRAM-based crossbar architecture is well-suited for these parallel accumulations, dynamically updating class $\mathcal{HV}$s and enabling efficient, on-the-fly classifications through simple binary similarity checks with minimal data movement during inference. IMC components handle the addition (step ④) and binarization (step ⑤) processes, leveraging Digital-to-Analog Converters (DACs), Analog-to-Digital Converters (ADCs) reported in [7], and ReRAM-based crossbar [8]. DACs are used to program each ReRAM cell to store individual bits of the wired-negated Hadamard $\mathcal{HV}$s, and to apply *read voltage* to each row of the crossbar for accumulation. The addition operation utilizes Ohm's Law to convert conductance values into currents and Kirchhoff's Current Law to sum these currents along the columns. The accumulated values are then binarized using ADCs to produce the final $\mathcal{HV}$.

III. Evaluation Results

We evaluate the performance and cost-efficiency of the proposed **AddOn-IHD** compared to the baseline HDC (Fig. 1(a)). Table I compares the accuracy and software simulation runtime of **AddOn-IHD** and the baseline for different datasets. The results demonstrate that the proposed approach achieves similar or, in many cases, superior accuracy compared to the baseline. For runtime, **AddOn-IHD** outperforms the baseline in most cases for D=4096.

Table II compares the ASIC hardware cost of generating a single pixel $\mathcal{HV}$ using the baseline and **AddOn-IHD** architecture in $45nm$ technology. The results indicate that **AddOn-IHD** surpasses the baseline design by up to 33%, 60%, 69%, 81%, and 70% improvements in critical path latency, area, power consumption, energy consumption, and area×delay, respectively. Table II also reports the iso-accuracy performance (with respect to MNIST case in Table I) of both architectures. To achieve the same 0.89 accuracy as **AddOn-IHD**, the baseline requires increasing the vector size from D=4096 to D=5120. While this adjustment allows the baseline to match **AddOn-IHD** in accuracy, it comes at the cost of increased hardware complexity, further emphasizing **AddOn-IHD**'s efficiency. Additionally, Table III presents the hardware cost of the remaining addition-only operations in IMC.

TABLE I
Classification Accuracy (5 epochs) and Inference Runtime

		D=4096		D=1024	
Dataset	**Encoding**	**Accuracy**	**Time**(s)	**Accuracy**	**Time**(s)
MNIST [9]	Baseline	0.88	4.804	0.86	3.138
	AddOn-IHD	**0.89**	**3.188**	**0.87**	**3.015**
PneumoniaMNIST [10]	Baseline	0.89	0.417	0.85	0.234
	AddOn-IHD	**0.94**	**0.283**	**0.93**	**0.229**
RetinaMNIST [11]	Baseline	0.51	0.079	0.48	0.059
	AddOn-IHD	**0.52**	0.079	**0.50**	**0.050**
BreastMNIST [12]	Baseline	0.77	0.028	0.77	0.022
	AddOn-IHD	**0.80**	0.031	**0.79**	0.022

TABLE II
Breakdown Hardware Cost of One Pixel $\mathcal{HV}$: ASIC, Steps 1&2&3

Encoding Approach	$\mathcal{HV}$ **length** (D)	**CPL*** (ns)	**Area** (μm^2)	**Power**✣ (mW)	**Energy** (nJ)	**Area×Delay** ($\mu m^2 \times ns$)
Baseline	256	0.45	445	1.56	0.18	200.25
AddOn-IHD		**0.30**	**224**	**0.65**	**0.05**	**67.20**
Baseline	512	0.44	528	1.77	0.40	232.32
AddOn-IHD		**0.30**	**240**	**0.67**	**0.10**	**72.00**
Baseline	1024	0.44	588	1.97	0.89	258.72
AddOn-IHD		**0.30**	**290**	**0.73**	**0.22**	**87.00**
Baseline	4096	0.44	710	2.28	4.11	312.40
AddOn-IHD		**0.33**	**313**	**0.76**	**1.03**	**103.29**
Baseline	iso-ac.	0.45	786	2.46	5.67	353.70
AddOn-IHD		**0.33**	**313**	**0.76**	**1.03**	**103.29**

Synopsys Design Compiler-$45nm$ || *:Critical Path Latency. || ✣: Power at max. freq.

TABLE III
Breakdown Hardware* Cost of One Pixel $\mathcal{HV}$: IMC, Steps 4&5

D	**Latency** (ns)	**Area** (mm^2)	**Power** (W)	**Energy** (nJ)	IMC Parameters [8]
256✣	20.00	0.02	0.13	2.68	•Read Volt.=$1V$ •Write Volt.=$2V$
512✣		0.02	0.26	5.37	•Read Pulse Width=$1ns$
1024✣		0.02	0.53	10.73	•Write Pulse Width=$20ns$
4096▲		0.11	2.14	42.95	•G_{ON}= 5e-4 •G_{OFF}=1.5e-8
iso-ac.★		0.14	2.68	53.69	•R_{sense}=$2000Ohm$

*NeuroSim [13] $45nm$ || ✣1, ▲4, and ★5 of 1024×1024 crossbars.

During inference, IMC enables operations to be performed near the associative memory where the class $\mathcal{HV}$s reside, eliminating the need for additional deployment steps. This configuration minimizes read-write overhead and enhances the efficiency of similarity calculations between the incoming query $\mathcal{HV}$ versus class $\mathcal{HV}$s in memory.

IV. Conclusion

This work introduced **AddOn-IHD**, a novel framework for hyperdimensional computing (HDC) that leverages unary computing and Hadamard processing to overcome the limitations of pseudo-random hypervector ($\mathcal{HV}$) generation and complex encoding steps in conventional HDC architectures. By replacing pseudo-random sources with orthogonal transformations through Hadamard and utilizing unary vectors, **AddOn-IHD** achieves enhanced efficiency and competitive classification accuracy with an addition-only encoding approach utilizing in-memory processing.

V. Acknowledgment

This work is supported in part by the National Science Foundation under grants #2019511, #2339701, and a generous gift from NVIDIA.

References

[1] P. Kanerva, *Sparse distributed memory*. MIT press, 1988.
[2] A. Rahimi, P. Kanerva, and J. M. Rabaey, "A robust and energy-efficient classifier using brain-inspired hyperdimensional computing," in *ISLPED*, 2016.
[3] M. Moghadam, S. Aygun, F. S. Banitaba, and M. H. Najafi, "All you need is unary: End-to-end unary bit-stream processing in hyperdimensional computing," in *ISLPED*, 2024.
[4] S. Aygun, M. H. Najafi, and M. Imani, "A linear-time, optimization-free, and edge device-compatible hypervector encoding," in *DATE'23*, 2023, pp. 1–2.
[5] P. Shlichta, "Higher-dimensional hadamard matrices," *IEEE Trans. Inf. Theory*, vol. 25, no. 5, pp. 566–572, 1979.
[6] D. Wu, J. Li, Z. Pan, Y. Kim, and J. S. Miguel, "uBrain: a unary brain computer interface," in *ISCA*, 2022.
[7] C. Liu, K. Wu, H. Liu, H. Jin, X. Liao, Z. Duan, J. Xu, H. Li, Y. Zhang, and J. Yang, "A ReRam-based processing-in-memory architecture for hyperdimensional computing," *IEEE TCAD*, vol. 44, 2024.
[8] Y. Deng, P. Huang, B. Chen, X. Yang, B. Gao, J. Wang, L. Zeng, G. Du, J. Kang, and X. Liu, "RRAM crossbar array with cell selection device: A device and circuit interaction study," *IEEE Trans. on Electron Dev.*, vol. 60, no. 2, 2012.
[9] Y. Lecun, "Mnist," https://api.openml.org/d/554, accessed: 2024-11-22.
[10] D. S. Kermany, M. Goldbaum, W. Cai, C. C. Valentim, H. Liang, S. L. Baxter, A. McKeown, G. Yang, X. Wu, F. Yan *et al.*, "Identifying medical diagnoses and treatable diseases by image-based deep learning," *cell*, vol. 172, no. 5, pp. 1122–1131, 2018.
[11] R. Liu, X. Wang, Q. Wu, L. Dai, X. Fang, T. Yan, J. Son, S. Tang, J. Li, Z. Gao *et al.*, "Deepdrid: Diabetic retinopathy—grading and image quality estimation challenge," *Patterns*, vol. 3, no. 6, 2022.
[12] W. Dhabyani, M. Gomaa, H. Khaled, and A. Fahmy, "Dataset of breast ultrasound images," *Data in Brief'20*.
[13] P. Chen, X. Peng, and S. Yu, "NeuroSim: A circuit-level macro model for benchmarking neuro-inspired architectures in online learning," *IEEE TCAD*, vol. 37, 2018.

Authorized licensed use limited to the terms of the applicable license agreement with IEEE. Restrictions apply.